

Create and Deploy an Azure Function App Using Visual Studio (Step by Step)

The following steps assume:

- Visual Studio 2022 (or later)
 - .NET 7 / .NET 8
 - Azure subscription
 - Background in ASP.NET Core (C#)
-

Part 1: Prerequisites

1. Install Required Components

Open **Visual Studio Installer** and ensure:

- **ASP.NET and web development** workload is installed
- **Azure development** workload is installed
- .NET 7 / .NET 8 SDK is installed

Restart Visual Studio after installation.

Part 2: Create Azure Function App in Visual Studio

Step 1: Create a New Project

1. Open **Visual Studio**
 2. Click **Create a new project**
 3. Search for **Azure Functions**
 4. Select **Azure Functions**
 5. Click **Next**
-

Step 2: Configure Project

1. Project name: SampleFunctionApp
2. Location: choose your folder

3. Solution name: same or different
 4. Click **Next**
-

Step 3: Select Runtime & Options

1. **Target Framework:**
 - .NET 8 (Isolated) (Recommended)
 - or .NET 6 / 7 if required
2. **Azure Functions version:** v4
3. **Authorization level:** Function (for HTTP trigger)
4. **Storage account:**
 - Select **Storage Emulator / Azurite** (for local dev)
5. Click **Create**

Visual Studio will generate a Function App project.

Part 3: Understand the Generated Project Structure

Key files:

- Program.cs – entry point (similar to ASP.NET Core)
 - local.settings.json – local configuration (not deployed)
 - Function1.cs – sample function
 - host.json – global function settings
-

Part 4: Create an HTTP Trigger Function

Step 4: Add New Function

1. Right-click the project → **Add** → **New Azure Function**
2. Select **HTTP trigger**
3. Function name: HelloFunction

4. Authorization: Function

5. Click **Add**

Sample Function Code (C# – Isolated)

```
using Microsoft.Azure.Functions.Worker;
```

```
using Microsoft.Azure.Functions.Worker.Http;
```

```
using Microsoft.Extensions.Logging;
```

```
using System.Net;
```

```
public class HelloFunction
```

```
{
```

```
    private readonly ILogger _logger;
```

```
    public HelloFunction(ILoggerFactory loggerFactory)
```

```
    {
```

```
        _logger = loggerFactory.CreateLogger<HelloFunction>();
```

```
    }
```

```
[Function("HelloFunction")]
```

```
public HttpresponseData Run(
```

```
    [HttpTrigger(AuthorizationLevel.Function, "get", "post")] HttpRequestData req)
```

```
{
```

```
    _logger.LogInformation("HelloFunction triggered");
```

```
    var response = req.CreateResponse(HttpStatusCode.OK);
```

```
    response.WriteString("Hello from Azure Functions!");
```

```
    return response;
```

```
}  
  
}
```

Part 5: Run Azure Function Locally

Step 5: Start Local Execution

1. Press **F5**
 2. Azure Functions Core Tools will start
 3. Console window shows the local URL:
 4. `http://localhost:7071/api/HelloFunction`
 5. Open browser or Postman and test the endpoint
-

Part 6: Create Azure Function App in Azure (from Visual Studio)

Step 6: Publish from Visual Studio

1. Right-click the project → **Publish**
 2. Select **Azure**
 3. Select **Azure Function App (Windows)** or **Linux**
 4. Click **Next**
-

Step 7: Create New Azure Resources

1. Click **Create new**
2. Fill in details:
 - **Function App name:** unique name
 - **Subscription**
 - **Resource Group:** new or existing
 - **Hosting Plan:** Consumption (recommended)
 - **Storage Account:** new or existing

- **Region**

3. Click **Create**

Visual Studio provisions resources and publishes the app.

Part 7: Verify Deployment in Azure Portal

Step 8: Test Function in Azure

1. Go to **Azure Portal**
 2. Open **Function App**
 3. Click **Functions** → **HelloFunction**
 4. Select **Get Function URL**
 5. Test in browser or Postman
-

Summary Flow

Visual Studio

- Create Azure Function Project
- Run locally
- Publish to Azure
- Verify in Portal
- Monitor via App Insights